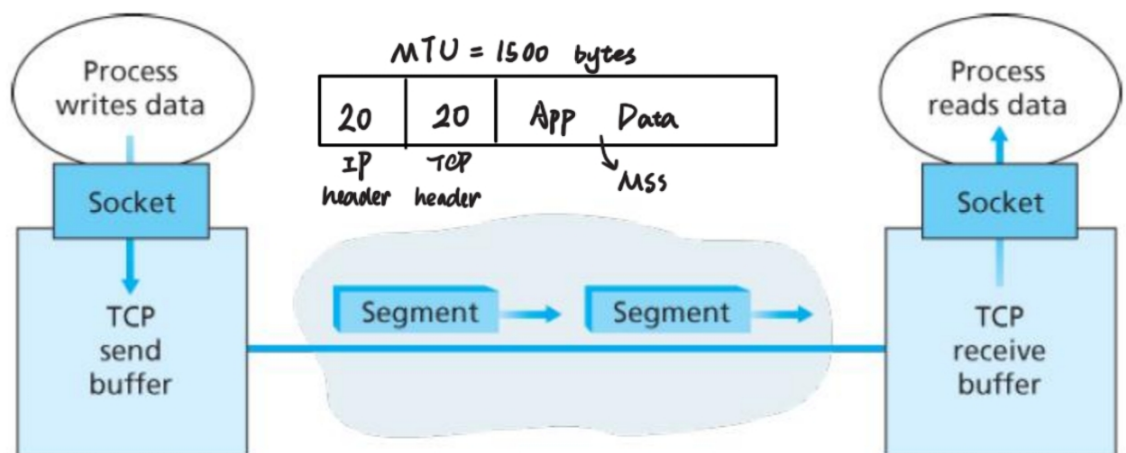


lecture 8 (Transport Layer -- TCP)

• 特点：

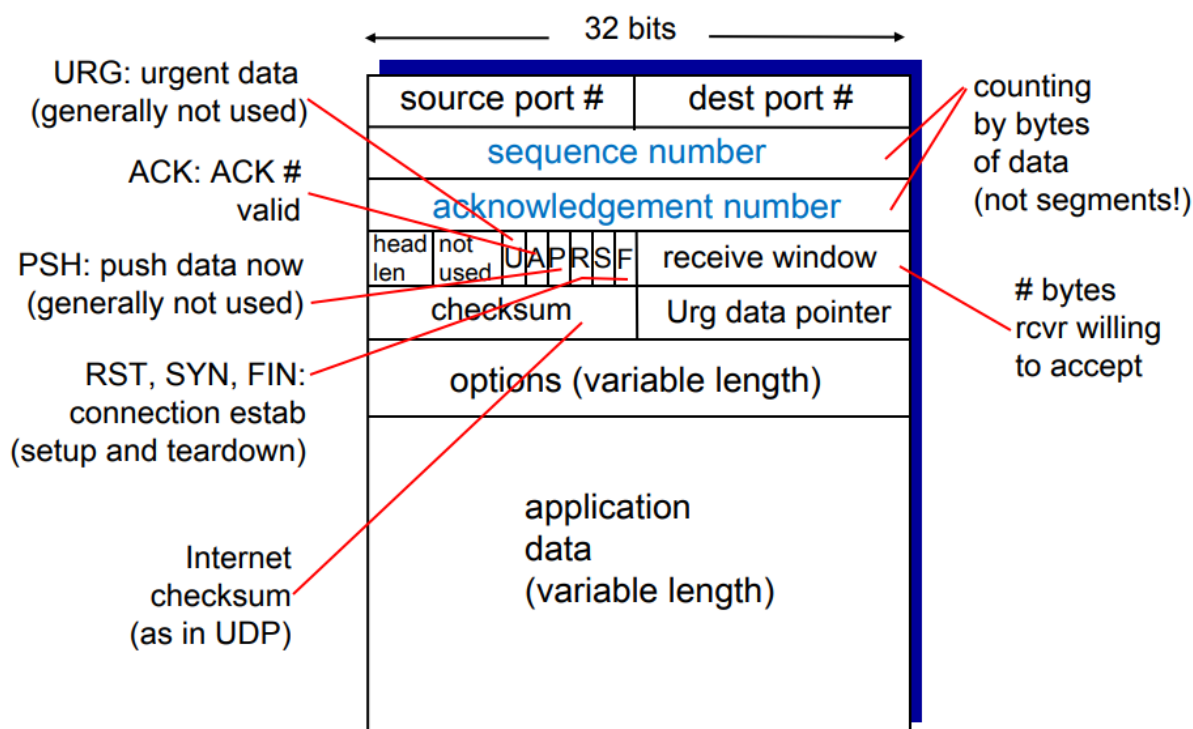
- 点对点 (Point-to-Point) :
 - TCP是一种点对点协议，只有一个发送方和一个接收方。
 - 网络元素之间没有缓冲区或变量。
- 可靠、有序的字节流 (Reliable, In-Order Byte Stream) :
 - 数据以字节流的形式传输，没有明确的消息边界。
 - 序号 (Seq#) 和确认号 (Ack#) 以字节为单位，而不是以数据包为单位。
- 管道化 (Pipelined) :
 - TCP利用拥塞控制和流量控制机制设置窗口大小。
- 全双工数据 (Full Duplex Data) :
 - 同一连接中支持双向数据流。
 - MSS (最大段大小) : 规定单个段的最大字节数。
- 面向连接 (Connection-Oriented) :
 - 通过握手 (控制消息交换) 建立连接，初始化发送方和接收方的状态。
- 流量控制 (Flow Controlled) :
 - 确保发送方不会压垮接收方。



- 发送方 (Sender) :
 - 应用程序通过Socket将数据写入TCP发送缓冲区。
 - TCP从发送方的缓冲区中抓取数据块 (chunks of data)，将数据分成段 (Segments)，并添加TCP/IP头部。
 - MSS (Maximum Segment Size) : 单个段的最大数据量，通常为1460字节。
 - MTU (Maximum Transmission Unit) : 链路层帧的最大大小，通常为1500字节。

- 包括应用层数据和TCP/IP头部（通常为40字节）。
- 段通过链路传输。
- 接收方（Receiver）：
 - TCP在接收端从链路中接收段，将其放入TCP接收缓冲区。
 - 应用程序从接收缓冲区读取数据。
- segment structure

TCP segment structure

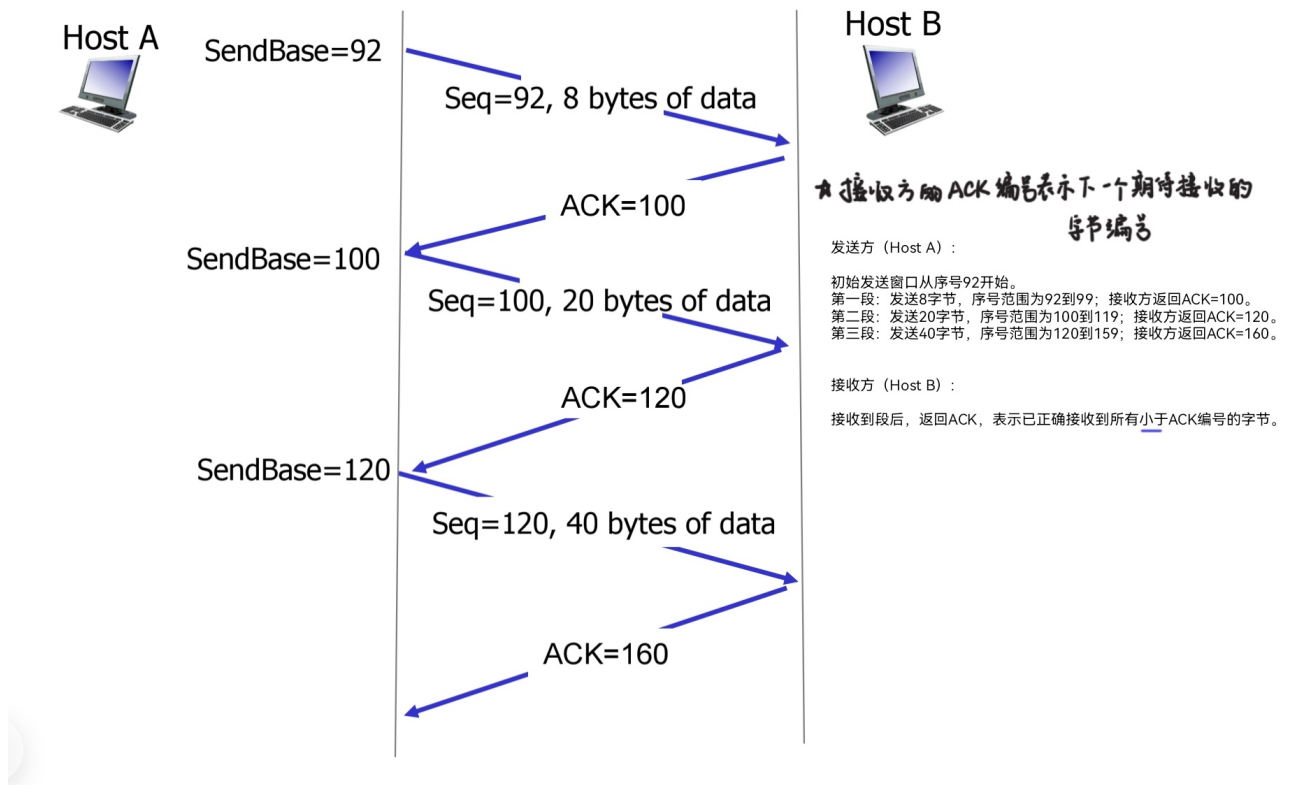


- 源端口号（Source Port #）和目标端口号（Dest Port #）：标识发送方和接收方的应用程序
- 序号（Sequence Number）：表示当前传输段中第一个字节的字节流编号
- 确认号（Acknowledgement Number）：表示接收方期待的下一个字节编号
- 接收窗口（Receive Window）：表示接收方愿意接收的字节数（接收方的缓冲区可用空间），用于流量控制
- 校验和（Checksum）：用于验证段数据的完整性
- 控制位：用于管理连接的建立、维护和释放
 - URG：紧急数据标志。
 - ACK：有效确认标志。
 - PSH：立即推送数据标志。
 - RST, SYN, FIN：连接建立和释放相关标志。

- **选项字段 (Options)**：可变长度，用于扩展功能。
- TCP 将数据视为连续的字节流，而不是单独传输的段或数据包。
- TCP 的序号用于标识字节流中的字节，而不是段的编号。



- 每个段的序号是该段中 **第一个字节在字节流中的位置**。
- TCP 以字节为单位传输数据，序号确保接收方能够正确组装字节流。
- 即使段的大小不同，序号仍然可以连续跟踪字节流。
- 例如：
 - 字节流从序号 0 开始。
 - 第一个段包含字节 0 到 999，则序号为 0。
 - 第二个段包含字节 1000 到 1999，则序号为 1000。
- ACK 表示接收方期待的下一个字节流编号。
 - 它是累积确认，表示所有小于当前 ACK 编号的字节已被正确接收
 - 如果接收方已接收到字节编号 0 到 535 和 900 到 1000：ACK = 536，表示接收方期待的下一个字节是 536
- TCP 规范未定义接收方如何处理乱序段。
 - 实现者可以选择：
 - 缓存乱序段，等待按序交付。
 - 丢弃乱序段，等待发送方重传。
- TCP 的初始序号是随机选择的
 - 目的：
 - 增强安全性，防止旧段被错误地识别为当前段
 - 初始序号由实现者决定，而非固定值
-

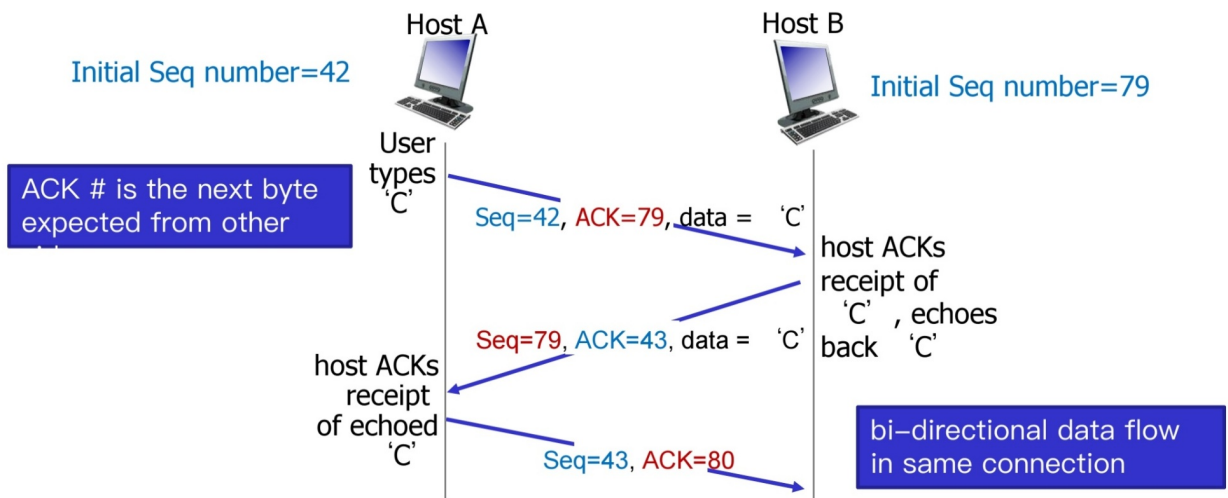


• Telnet Case Study

- User types a character at host A, and host A sends the character to host B
- Host B sends back a copy of the character
- Host A displays the character on user's screen

用户在Host A键入字符“C”, Host A将该字符发送到Host B。
 Host B接收字符后, 将其回显给Host A。
 序号和ACK更新:

Host A发送字符“C”, 序号为42, ACK=79 (期待Host B发送的下一个字节)。
 Host B回显字符“C”, 序号为79, ACK=43。
 Host A确认收到回显字符, 序号为43, ACK=80。



- 双向数据流:
 - 同一连接支持双向数据流
 - 每一方向都有独立的序号和ACK编号
- 序号与ACK的关系:
 - 序号标识当前段的起点字节
 - ACK编号表示接收方期待的下一字节

- Round-trip time estimation

- TCP超时设置

- 如何设置TCP超时值？

- 超时值应 大于RTT，但RTT是动态变化的。
 - 过短的超时值：可能导致过早超时，触发不必要的重传。
 - 过长的超时值：会对丢包反应过慢，降低协议效率。

- RTT的估算方法

- SampleRTT:

- 测量从段传输到ACK接收的时间。
 - 应忽略重传的时间（避免重复计算）。

- EstimatedRTT:

- 平滑处理，用于估算RTT，避免直接使用单次测量值。
 - 通过多次测量的平均值计算，减小SampleRTT的波动。
 - 公式： $\text{EstimatedRTT} = (1-\alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$

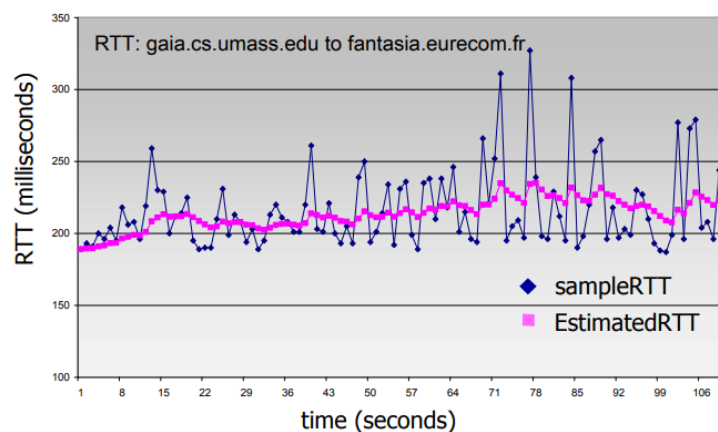
- 公式含义：

- 基于 指数加权移动平均（EWMA） 的方法。
 - 新的测量值（SampleRTT）占比由权重 α 决定。
 - α 的典型值为 0.125，即新测量值占12.5%的权重。

- 公式特点：

- 更早的测量值对当前估算的影响逐渐减弱（指数衰减）
 - 降低SampleRTT的波动对EstimatedRTT的影响

- 从 gaia.cs.umass.edu 到 fantasia.eurecom.fr 的RTT:



- SampleRTT的波动：可见实际测量值存在较大波动。
 - EstimatedRTT的平滑效果：通过指数加权平均，降低了SampleRTT的波动对RTT估算的影响。

- RTT变动性 (DevRTT)

- 公式： $DevRTT = (1 - \beta) \times DevRTT + \beta \times |SampleRTT - EstimatedRTT|$
 - 表示SampleRTT与EstimatedRTT之间的偏差
 - 通过指数加权平均计算，典型值 $\beta = 0.25$
- 作用：
 - 表示RTT的变动幅度。
 - 变动幅度越大，超时间隔需要更大的安全边际。
- TCP超时间隔计算
 - 公式： $TimeoutInterval = EstimatedRTT + 4 \times DevRTT$
 - 超时间隔由估算RTT和安全边际safety margin（4倍DevRTT）组成。
 - 安全边际的大小与EstimatedRTT的变动性直接相关。
 - 公式解释：
 - 如果RTT波动较大，安全边际增加，超时间隔会更长。
 - 确保超时设置既不过短也不过长，优化传输效率。

• Reliable data transfer

- TCP如何保证可靠数据传输
 - 基于不可靠的IP服务：
 - TCP在IP协议之上构建可靠传输服务rdt
 - IP协议本身不提供可靠性，可能导致数据丢失或乱序
 - 关键机制：
 - 管道化段传输（Pipelined Segments）：
 - 使用窗口机制和 SendBase 变量控制未确认段的范围。
 - 累积ACK（Cumulative ACKs）：
 - 接收方确认到目前为止按序接收的最高字节。
 - 简化发送方的ACK处理逻辑。
 - 单定时器（Single Retransmission Timer）：
 - 发送方只为窗口内最早未确认段维护一个定时器。
 - 定时器用于监控窗口内最早未确认段的超时情况。
 - 超时间隔由 EstimatedRTT 和 DevRTT 决定，确保设置合理的重传时间。
 - 触发重传的事件：
 - 超时事件（Timeout Events）：发送方未收到ACK时触发重传。
 - 重复ACK（Duplicate ACKs）：接收方收到乱序段时发送重复ACK，通知发送方丢失的段。
- 发送方事件处理
 - 数据接收事件（从应用层接收到数据时）：
 - 创建段并分配序号

- 序号为段中第一个字节的字节流编号
- 如果定时器未运行，则启动定时器
- 超时事件（定时器到期时）：
 - 重传导致超时的段（窗口内最早未确认段）。
 - 重新启动定时器。
- ACK接收事件（如果ACK确认了窗口内的未确认段）：
 - 更新窗口起始点（将 SendBase 移动到下一个未确认段）。
 - 如果仍有未确认段，重新启动定时器。
- TCP sender events:

```
NextSeqNum=InitialSeqNumber
SendBase=InitialSeqNumber
```

```
loop (forever) {
    switch(event)
```

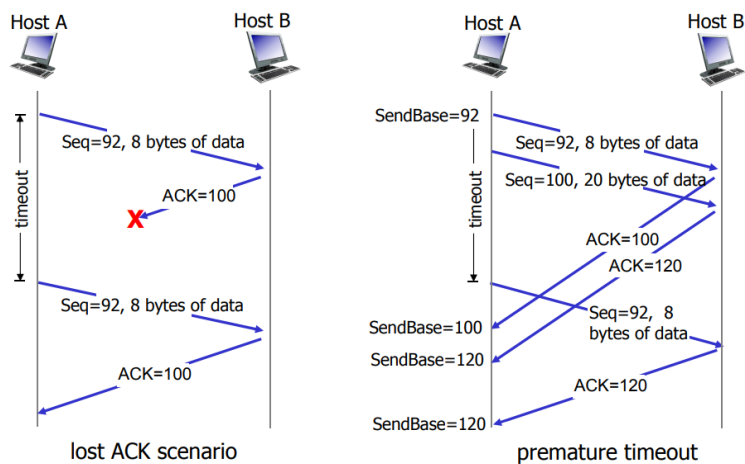
```
event: data received from application above
    create TCP segment with sequence number NextSeqNum
    if (timer currently not running)
        start timer
    pass segment to IP
    NextSeqNum=NextSeqNum+length(data)
    break;
```

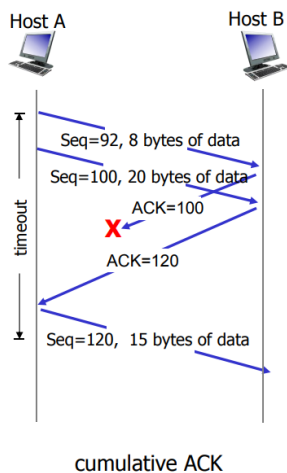
```
event: timer timeout
    retransmit not-yet-acknowledged segment with
        smallest sequence number
    start timer
    break;
```

```
event: ACK received, with ACK field value of y
    if (y > SendBase) {
        SendBase=y
        if (there are currently any not-yet-acknowledged segments)
            start timer
    }
    break;
```

```
} /* end of loop forever */
```

TCP: retransmission scenarios





• 接收方事件与行为

• 按序到达且已确认的段：

- 事件：到达的段序号为接收方期待的序号，且之前的所有数据段已确认
- 行为：接收方使用 **延迟ACK (Delayed ACK) 机制**：
 - 等待最多500毫秒，查看是否有其他数据段到达
 - 如果没有其他数据段到达，则发送ACK

• 按序到达且有待确认的段：

- 事件：到达的段序号为接收方期待的序号，但之前有未确认的数据段。
- 行为：立即发送一个 **累积ACK (Cumulative ACK)**，确认所有按序接收的数据段

• 乱序段到达：

- 事件：到达的段序号高于接收方期待的序号，且检测到数据段之间存在间隙
- 行为：立即发送一个 **重复ACK (Duplicate ACK)**，指示期待的下一个字节序号

• 填补间隙的段到达：

- 事件：到达的段可以部分或完全填补间隙。
- 行为：如果段从间隙的低端开始，则立即发送ACK。

• 双倍超时机制 (Double Timeout Interval)

- 每次TCP触发重传，都会将 **下一次超时间隔** 设置为 **前一次超时间隔的两倍**
- 原因：
 - 防止过于频繁的重传导致网络拥塞。
 - 避免在高延迟网络中浪费资源。
- 当在下面两种事件之后启动定时器（即：上层应用传来数据，或收到ACK）时：
 - 超时间隔 (TimeoutInterval) 由最新的 EstimatedRTT 和 DevRTT 值计算得出。

• TCP Fast Retransmit (TCP快速重传)

- 超时间隔通常较长：
 - 在超时间隔内，丢失的数据包可能需要较长时间才能被重传。
 - 这种延迟可能影响传输效率。

- 通过重复ACK检测丢失的段：
 - 如果发送方连续发送多个段，其中一个段丢失，接收方可能会多次发送重复ACK。
 - 重复ACK表明接收方期待的是丢失的段。
- 快速重传机制：
 - 如果发送方连续收到 3个重复ACK：立即重传窗口内 最小序号的未确认段，无需等待超时间隔
 - 这是因为丢失的段很可能是未被确认的段。

TCP fast retransmit

```

NextSeqNum=InitialSeqNumber
SendBase=InitialSeqNumber

loop (forever) {
  switch(event)

    event: data received from application
      create TCP segment with sequence
      if (timer currently not running)
        start timer
      pass segment to IP
      NextSeqNum=NextSeqNum+length(data)
      break;

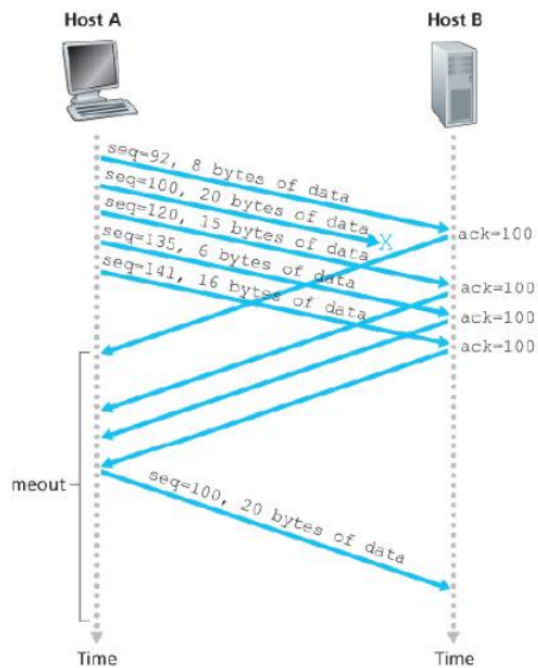
    event: timer timeout
      retransmit not-yet-acknowledged segment
      smallest sequence number
      start timer
      break;

    event: ACK received, with ACK field value of y
      if (y > SendBase) {
        SendBase=y
        if (there are currently any not yet
            acknowledged segments)
          start timer
      }
      else { /* a duplicate ACK for already ACKed
              segment */
        increment number of duplicate ACKs
        received for y
        if (number of duplicate ACKs received
            for y==3)
          /* TCP fast retransmit */
          resend segment with sequence number y
      }
      break;
  }
} /* end of loop forever */

```

ACK接收事件：

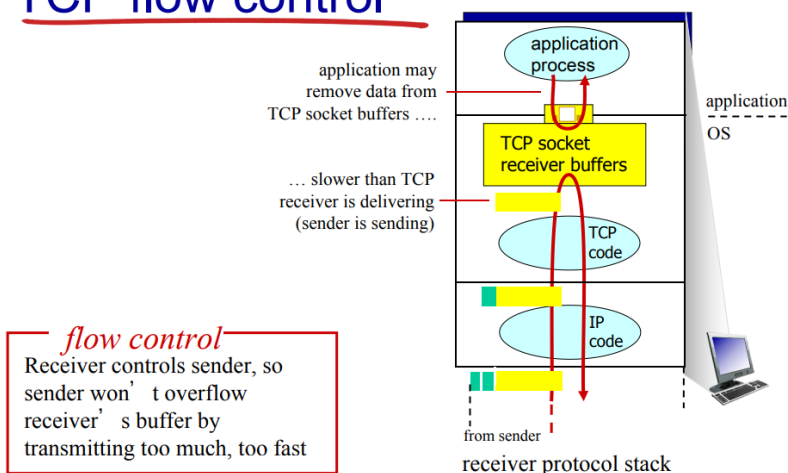
- 如果 ACK 的值大于当前窗口起点 SendBase：
 - 更新 SendBase，表示窗口内的段已被确认。
 - 如果仍有未确认段，启动定时器。
- 如果接收到重复ACK：
 - 增加该重复ACK的计数。
 - 如果重复ACK计数达到3：触发快速重传，发送未确认段中序号最小的段。



fast retransmit after sender receipt of triple duplicate ACK

• Flow control

TCP flow control



• 应用层与TCP缓冲区：

- 应用程序从TCP接收缓冲区中移除数据。
- 但应用程序移除数据的速度可能慢于TCP接收数据的速度。

• 流量控制机制：

- 接收方控制发送方，防止发送方传输过多数据，导致接收缓冲区溢出。

• 流量控制的意义：

- TCP的流量控制机制通过接收方的反馈确保发送方不会超速发送数据。
- 接收方缓冲区的状态（已用空间和剩余空间）决定发送方的传输速度。

• 关键点：

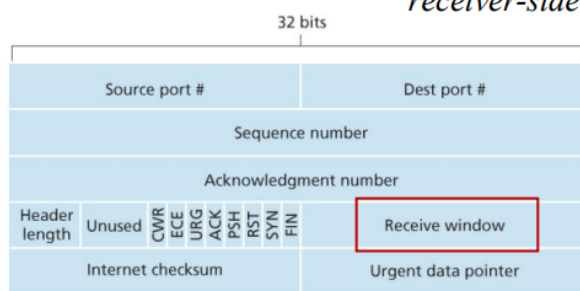
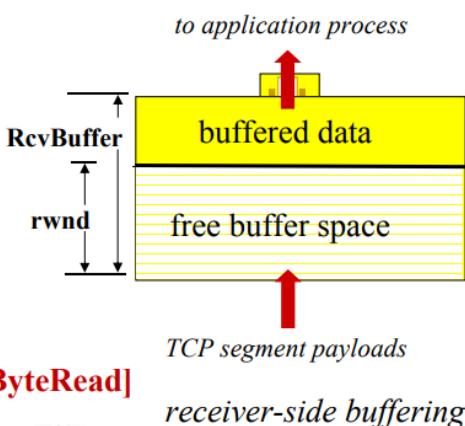
- 如果接收方缓冲区满，接收方会通知发送方暂停发送。
- TCP通过接收窗口（Receive Window, rwnd）实现流量控制。

TCP flow control

Receiver “advertises” free buffer space by including **rwnd** value in TCP header of receiver-to-sender segments

- ❖ **RcvBuffer** size set via socket options (typical default is 4096 bytes)
- ❖ many operating systems autoadjust **RcvBuffer**

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$



接收方的作用：

- 接收方通过TCP段的头部字段 rwnd 向发送方通告剩余的缓冲区空间。
 - 接收窗口的作用：
 - 动态调整机制：
 - 接收方缓冲区的大小可能因操作系统或应用程序需求而改变。
 - 这确保TCP能够适应不同的流量和负载条件。
 - 发送方根据 rwnd 限制未确认数据（in-flight data）的数量。

接收窗口的计算： $rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$

- RcvBuffer：接收缓冲区大小可通过 socket 选项设置（默认值 4096 字节），许多操作系统会根据应用层需求动态调整 RcvBuffer 的大小。
- LastByteRcvd：最后一个接收到的字节序号。
- LastByteRead：应用程序读取的最后一个字节序号。

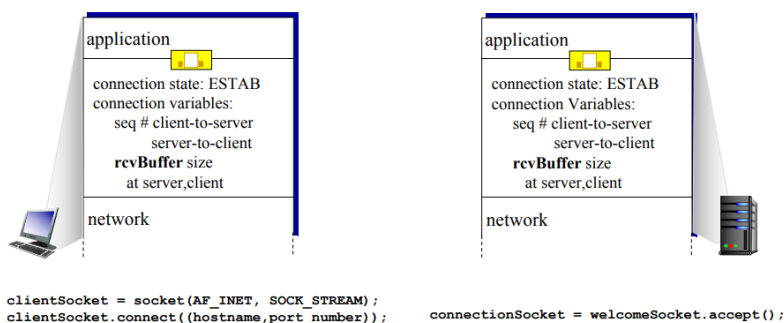
发送方限制未确认数据：

- 发送方根据接收方的 rwnd 限制飞行数据的数量： $LastByteSent - LastByteAcked \leq rwnd$
 - LastByteSent：发送方已发送的最后一个字节序号。
 - LastByteAcked：接收方已确认的最后一个字节序号。

- 目标：确保接收方的缓冲区不会溢出。
- 发送方的角色：
 - 发送方通过接收窗口信息动态调整发送速度。
 - 如果 `rwnd` 值较小，发送方会减少发送数据量。
- 关键点：
 - 流量控制的核心是接收方的反馈机制。
 - 发送方必须实时监控 `rwnd` 值，避免传输过多数据。

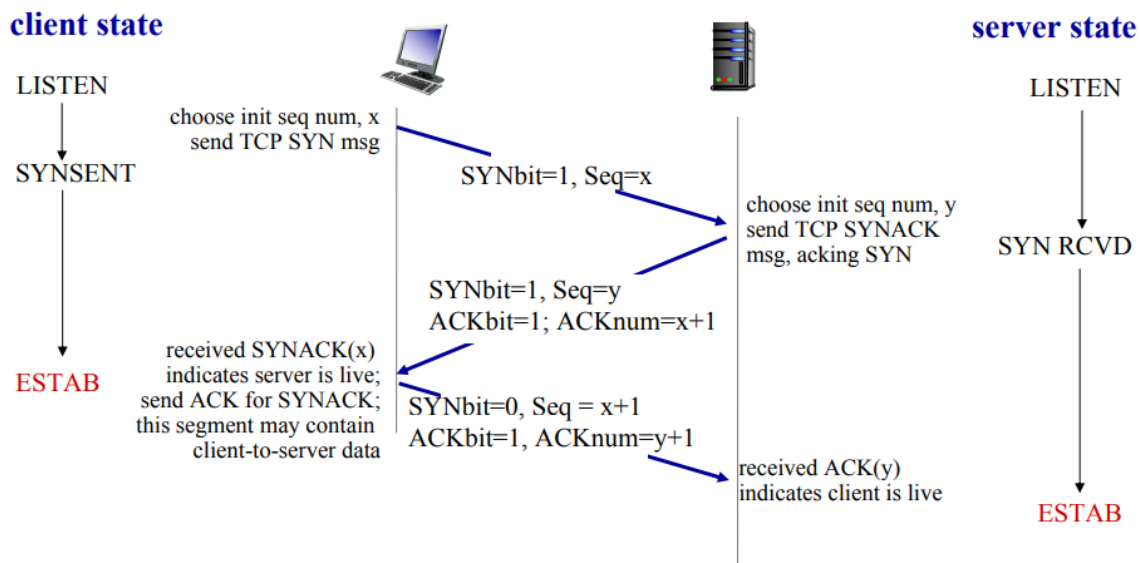
• Connection management

- 数据交换前的握手：
 - 在交换数据之前，发送方和接收方通过握手过程进行连接管理。
 - 握手的两个目标：
 - 确认双方愿意建立连接。
 - 就连接参数（如序号和缓冲区大小）达成一致。
 - 连接参数的重要性：
 - 序号用于标识字节流中的位置，确保数据按序传输。
 - 接收缓冲区大小决定了流量控制的能力。
- 连接状态和变量：
 - 连接状态：ESTAB（已建立连接）。
 - 连接变量：
 - 客户端到服务器的序号。
 - 服务器到客户端的序号。
 - 接收缓冲区的大小（`rcvBuffer`）。



- 客户端：使用 `socket(AF_INET, SOCK_STREAM)` 创建套接字并通过 `connect` 方法连接到服务器。
- 服务器：通过 `accept` 方法接受客户端连接。
- TCP 3-Way Handshake（三次握手具体流程）

TCP 3-way handshake



Once these three steps have been completed, the client and server hosts can send segments containing data to each other.

- In each of these future segments, SYNbit=0

• 客户端状态:

- LISTEN: 等待连接请求。
- SYNSENT: 发送SYN消息, 初始化序号。
- ESTAB: 收到服务器的SYN-ACK后, 发送ACK进行确认。

• 服务器状态:

- LISTEN: 等待连接请求。
- SYN RCVD: 收到客户端的SYN后, 发送SYN-ACK进行确认。
- ESTAB: 收到客户端的ACK后, 完成连接建立。

• 三次握手的消息内容:

- 第一步: 客户端发送 SYN 消息, 序号为 x。【初始化序号是随机选择的, 防止旧段被错误地识别。】
- 第二步: 服务器回复 SYN-ACK 消息, 序号为 y, 确认号为 x+1。【确认号确保消息按序传输。】
- 第三步: 客户端发送 ACK 消息, 确认号为 y+1。

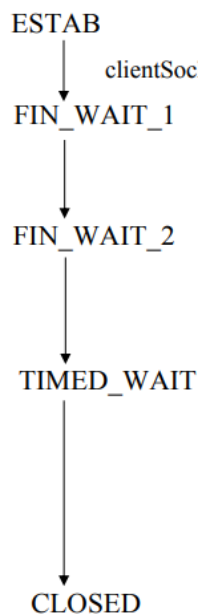
• 数据传输:

- 握手完成后, 客户端和服务端可以发送包含数据的段。数据段头部包含序号和确认号, 用于可靠性保证。
- 此时段的 SYN 位为0。

• TCP: Closing a Connection (TCP关闭连接)

TCP: closing a connection

client state



can no longer
send but can
receive data

wait for server
close

timed wait



FINbit=1, seq=x

ACKbit=1; ACKnum=x+1

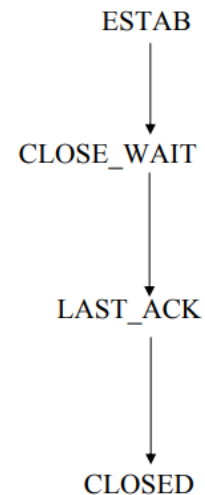
FINbit=1, seq=y

ACKbit=1; ACKnum=y+1

can still
send data

can no longer
send data

server state



The TIME_WAIT state lets the TCP client resend the final acknowledgment in case the ACK is lost.

• 客户端状态转换：

- **ESTAB**（已建立连接）：通过调用 `clientSocket.close()` 开始关闭连接。
- **FIN_WAIT_1**：发送 FIN 段，序号为 x，表示不再发送数据但可以接收数据。
- **FIN_WAIT_2**：收到服务器的 ACK 后，等待服务器发送关闭请求。
- **TIMED_WAIT**：收到服务器的 FIN 段后，发送确认（ACK），进入定时等待状态。
- **CLOSED**：定时等待结束，连接完全关闭。

• 服务器状态转换：

- **ESTAB**：连接已建立，仍可以发送数据。
- **CLOSE_WAIT**：收到客户端的 FIN 段后，发送 ACK。
- **LAST_ACK**：发送自己的 FIN 段，等待客户端的确认（ACK）。
- **CLOSED**：收到客户端的确认后，连接完全关闭。

• TIME_WAIT的作用：

- 确保客户端能够重传最后的确认（ACK），以防服务器没有收到。
- TIME_WAIT状态的存在是为了保证可靠性。如果最后的ACK丢失，服务器会重传FIN，客户端需要在TIME_WAIT期间能够处理该FIN并重传ACK。

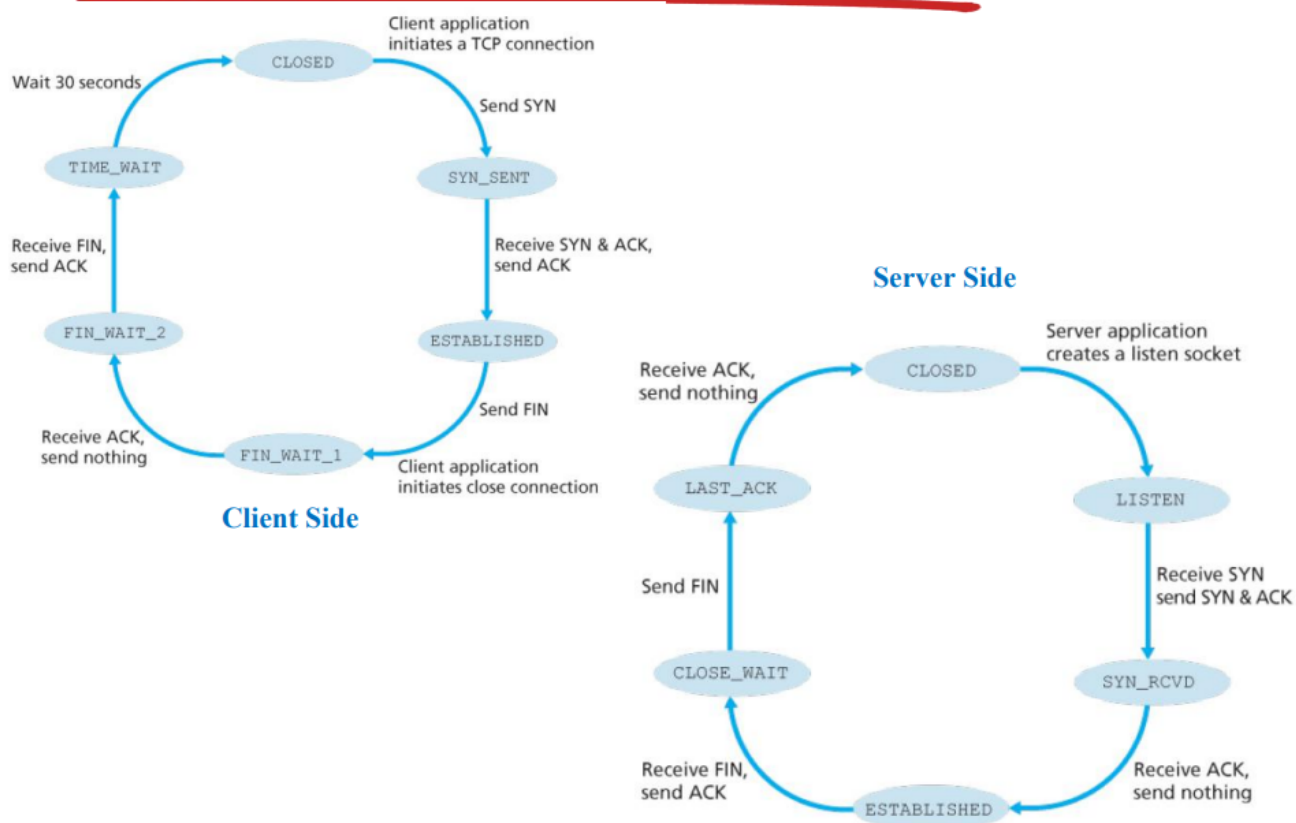
• 四次握手过程：客户端和服务端分别关闭自己的连接侧：

- 发送TCP段，FIN 位设置为1。
- 对收到的 FIN 段进行确认，发送 ACK。

• 为什么FIN和ACK不能在同一个消息中发送？

- 另一侧可能还有数据需要发送，因此不能直接发送 FIN，必须等待传输完成后关闭连接。
- TCP设计中，FIN和ACK分开处理，避免未完成的数据传输被强制中断。

TCP States



客户端状态：

- CLOSED：初始化状态。
- SYN_SENT：发送连接请求（SYN）。
- ESTABLISHED：连接建立后，进行数据传输。
- FIN_WAIT_1：发送 FIN，进入关闭连接过程。
- FIN_WAIT_2：等待服务器的关闭请求。
- TIME_WAIT：定时等待，确保最后的确认消息被正确接收。

服务器状态：

- CLOSED：初始化状态。
- LISTEN：等待连接请求。
- SYN_RCVD：收到客户端的连接请求，发送确认（SYN-ACK）。
- ESTABLISHED：连接建立后，进行数据传输。
- CLOSE_WAIT：收到客户端的关闭请求，发送确认。
- LAST_ACK：发送自己的关闭请求（FIN），等待客户端确认。

- **Reset Segment（复位段）**

- **复位段的触发条件：**

- 当主机接收到一个TCP段，其端口号或源IP地址与当前正在进行的连接不匹配时：
 - 主机发送一个特殊的复位段（RST），其中 RST 位设置为1。

- **复位段的内容：**

- 表示主机没有对应的套接字处理该段，要求发送方不要重发该段。

- **复位段的作用：**

- 防止错误的消息继续传输，释放资源。
 - 提供一种快速关闭连接的机制，避免浪费网络资源。

- **典型场景：**未匹配的连接请求。主机因资源问题需要立即释放连接。